Team: Keyboard Gremlins

Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform

API Service Contract

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Team Creation and Management

Service Name: Create Team Service

Description: Creates a new team for a specific event and automatically adds the creator as an approved member.

Inputs:

- teamName (string) – The name of the team (must be unique within the event).
- eventId (UUID) – The identifier of the event the team belongs to.
- currentUserId (UUID) – The identifier of the authenticated user creating the team.

Outputs:

- teamId (UUID) – Unique identifier of the created team.
- teamName (string) – Name of the team.
- eventId (UUID) – Event the team belongs to.
- createdByUserId (UUID) – ID of the user who created the team.
- createdAt (datetime) – Timestamp of creation.
- status (string) – Team status (e.g., "ACTIVE").

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams with a JSON body containing teamName and eventId.
- The request must include authentication (HTTP Basic Auth or JWT). The user ID is extracted from the security context.

- Preconditions:
    - Team name must not already exist for the same event.
    - The user must not already be an approved member of another team in the same event.
- On success, returns HTTP 201 Created with the team object.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Request to Join Team Service

Description: Allows a participant to request joining an existing team. The request is stored as a pending membership.

Inputs:

- teamId (UUID) – Identifier of the team to join (in the URL path).
- currentUserId (UUID) – Identifier of the authenticated user requesting to join.

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams/{teamId}/join-requests.
- The request must include authentication.
- Preconditions:
    - The team must exist and be active.

    - The user must not already have a pending or approved membership for this team.
    - The team must not have reached its maximum size (currently hardcoded to 4).
- On success, returns HTTP 201 Created with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Approve or Reject Join Request Service

Description: Allows the team creator to approve or reject a pending join request from another user.

Inputs:

- teamId (UUID) – Identifier of the team (in the URL path).
- userId (UUID) – Identifier of the user whose request is being decided (in the URL path).
- approve (boolean) – true to approve, false to reject (in the request body).
- currentUserId (UUID) – Identifier of the authenticated user (must be the team creator).

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a PUT request to /api/teams/{teamId}/join-requests/{userId} with a JSON body {"approve": true|false}.
- The request must include authentication.
- Preconditions:
    - The authenticated user must be the team creator.
    - The pending request must exist and have status PENDING.
    - If approving, the user must not already be an approved member of another team in the same event, and the team must still have capacity.

- On success, returns HTTP 200 OK with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Leave Team Service

Description: Allows a user to leave a team. If the user is an approved member, the membership status changes to "LEFT". If the user has only a pending request, the membership record is deleted. If the last approved member leaves, the team becomes inactive.

Inputs:

- teamId (UUID) – Identifier of the team to leave (in the URL path).
- currentUserId (UUID) – Identifier of the authenticated user leaving.

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/teams/{teamId}/members.
- The request must include authentication.
- Preconditions:
    - The user must be a member of the team (either pending or approved).
- On success, returns HTTP 204 No Content with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: View Team Members Service

Description: Retrieves the list of approved members of a team, including their names, emails, join dates, and team role (LEADER or MEMBER).

Inputs:

- teamId (UUID) – Identifier of the team (in the URL path).

Outputs:

- A list of member objects, each containing:
    - userId (UUID) – User identifier.
    - fullName (string) – Concatenated first and last name.
    - email (string) – User's email address.
    - joinedAt (datetime) – When the user joined.
    - role (string) – Either "LEADER" or "MEMBER".

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/{teamId}/members.
- Authentication may be required (depending on security configuration).

- Preconditions:
    - The team must exist.
- On success, returns HTTP 200 OK with the JSON array of members.
- On failure (e.g., team not found), returns an appropriate HTTP error with a message.

File Storage - Specs and Submission Upload/Download

Service Name: Upload Level File Service

Description: Allows an Admin to upload an input or resource file for a specific event level. The file is stored in Azure Blob Storage and the storage key is returned for database persistence.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path).
- levelId (long) - Identifier of the level (in the URL path). (previously documented as a generic string; it's the numeric levels.id)
- file (multipart/form-data) - The file to upload.

Outputs:

- id (string) - New. The generated levelfiles.id.
- storageKey (string) - Blob path persisted to levelfiles.storage_key: hackathons/{hackathonId}/levels/{levelId}/{filename}.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files with a multipart/form-data body containing fields named file and fileType.
- The request must include authentication (JWT). Admin role required.

- On success, returns HTTP 200 OK with id, storageKey, and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Level File Download URL Service

Description: Returns a time-limited presigned SAS URL for a participant or admin to download a level input file directly from Azure Blob Storage.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path).
- levelId (string) - Identifier of the level (in the URL path).
- filename (string) - The filename to retrieve (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files/{filename}.
- The request must include authentication (JWT). Admin or Participant role required.

- This endpoint does not query the database - it rebuilds the storage key deterministically from the path parameters. The filename supplied must exactly match the sanitised filename used at upload time.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Service Name: List Level Files Service

Description: Lists all files uploaded for a specific level. Used by the admin Levels page to render each level's file list.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- levelId (long) - Identifier of the level (in the URL path)

Outputs:

- A list of LevelFile objects, each containing:
  - id (long) - Database identifier
  - levelId (long) - The level this file belongs to
  - fileName (string) - Original filename

- storageKey (string) - Blob storage path
- fileType (string) - File type (JSON)
- fileSize (long) - Size in bytes
- contentType (string) - MIME type
- uploadedAt (datetime) - Upload timestamp

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of level files
- On failure, returns an appropriate HTTP error with a message

Service Name: Delete Level File Service

Description: Deletes a level file, removing both the blob and its metadata record.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- levelId (long) - Identifier of the level (in the URL path)

- fileId (long) - Identifier of the level file to delete (in the URL path)

Outputs:

- None (only HTTP status)

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files/{fileId}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 204 No Content with no body
- On failure, returns an appropriate HTTP error with a message

Service Name: Upload Solver File Service

Description: Allows an Admin to upload a solver file for a specific hackathon. Supports versioning so hotfix solver versions can be uploaded without overwriting previous ones. Automatically deactivates all previous solver versions for the hackathon before saving the new one as active - only one solver version can be active per hackathon at a time.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- file (multipart/form-data) - The solver file to upload
- uploadedBy (UUID) - The admin performing the upload (extracted from authentication context)
- notes (string) - Optional release notes for this solver version

Outputs:

- solverVersionId (string) - The generated solverversion.id
- storageKey (string) - The blob path persisted to solverversion.storage_key: hackathons/{hackathonId}/solver/v{version}/{filename}
- blobUrl (string) - The raw Azure blob URL
- version (string) - The version number that was uploaded

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/solver with a multipart/form-data body containing file, plus notes as optional request parameter
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with solverVersionId, storageKey, blobUrl, and version
- On failure, returns an appropriate HTTP error with a message

Service Name: Upload Branding Asset Service

Description: Allows an Admin to upload a branding asset such as a logo or banner image for a specific event.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- file (multipart/form-data) - The image file to upload

Outputs:

- storageKey (string) - The blob path of the uploaded asset
- blobUrl (string) - The raw Azure blob URL

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/branding with a multipart/form-data body containing a field named file
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with storageKey and blobUrl
- On failure, returns an appropriate HTTP error with a message

Service Name: Upload Problem Statement Service

Description: Uploads the problem statement PDF for a hackathon. The returned storageKey is saved to hackathon.problem_statement_storage_key, replacing any previous problem statement for this hackathon.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- file (multipart/form-data) - The uploaded PDF file (must be application/pdf)

Outputs:

- storageKey (string) - The blob path of the uploaded problem statement
- blobUrl (string) - The raw Azure blob URL

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/problem-statement with a multipart/form-data body containing a field named file
- The request must include authentication (JWT)
- Admin role required
- The file must be a PDF (content-type: application/pdf)
- On success, returns HTTP 200 OK with storageKey and blobUrl
- On failure (e.g., not a PDF, no file provided), returns an appropriate HTTP error with a message

Service Name: Get Problem Statement Download URL Service

Description: Returns a presigned SAS URL for downloading a hackathon's problem statement PDF.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes
- storageKey (string) - The blob storage key of the problem statement

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/problem-statement
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url and storageKey fields
- On failure (e.g., no problem statement uploaded), returns HTTP 404 Not Found

Service Name: Upload Submission Service

Description: Allows a Participant to submit a solution for a level in a single request - uploads both the solution output file and the zipped source code archive together, creates the submissions database record, and returns the generated submissionId.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- outputFile (multipart/form-data) - The solution output file (the artifact the solver grades)
- sourceFile (multipart/form-data) - A .zip archive of the source code
- levelId (short) - The level this submission is for

Outputs:

- submissionId (string) - The generated submissions.id
- outputStorageKey (string) - The blob path persisted to submissions.output_storage_key: submissions/{eventId}/{teamId}/levels/{levelId}/{submissionId}/output/{filename}
- sourceStorageKey (string) - The blob path persisted to submissions.source_code_storage_key, same shape with /source/ instead of /output/
- status (string) - The submission's status immediately after creation (currently "QUEUED")

- scoringRecordId (string) - The queue record identifier for the scoring job

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/events/{eventId}/teams/{teamId}/submissions with a multipart/form-data body containing outputFile and sourceFile, plus levelId as a request parameter
- The request must include authentication (JWT)
- Participant role required
- The platform never executes participant code. Only the output file is passed to the solver
- On success, returns HTTP 200 OK with submissionId, outputStorageKey, sourceStorageKey, status, and scoringRecordId
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Submission Output Download URL Service

Description: Returns a presigned SAS URL for downloading a submission output file from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)
- filename (string) - The output filename (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/output/{filename}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Source Archive Download URL Service

Description: Returns a presigned SAS URL for downloading a source code archive from Azure Blob Storage. Admin only - used for post-event auditing to verify that the submitted output is reproducible by the team's code.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)
- filename (string) - The archive filename (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/source/{filename}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Scoring Log Download URL Service (Single Submission)

Description: Returns a presigned SAS URL for downloading a single submission's scoring log from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- levelId (string) - Identifier of the level (in the URL path)
- submissionId (string) - Identifier of the submission (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/levels/{levelId}/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Admin Event Management

Service Name: Create Event Service

Description: Creates a new hackathon event with specific constraints

Inputs:

- EventRequest object payload in the request body that contains:
    - Name (string) - The title of the event.
    - registrationKey (string) - The optional passkey for private events.
    - teamSizeLimit (short) - The maximum number of participants per team.
    - startDateTime (OffsetDateTime) - The starting time and date of the event.
    - Duration (integer) - the total duration of an event in hours.
    - Description (string) - Information about the specific hackathon event.
    - Visibility (string) - The visibility of the hackathon event to the participants.
    - Status (string) - The lifecycle of an event, it can be ACTIVE or INACTIVE as an example.

Output:

- eventId (UUID) - Automatically generated unique database identifier for the event.
- createdByUserId (UUID) - The identifier of the admin that created the event.
- Name (string) - The configured title of the event.
- registrationKey (string) - The password for the event or null if Status is not private.

- teamSizeLimit (short) - The configured team size constraint for the event.
- startDateTime (OffsetDateTime) - The configured start date for the event.
- Duration (int) - The configured duration of the event.
- Description (string) - The configured description of the event.
- Visibility (string) - The configured visibility of the event.
- Status (string) - The configured status of the event.

Usage / Interaction Rules:

- Clients must send a POST request to /api/admin/events.
- The body of the request must include a valid JSON object matching the properties expected by EventRequest.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 201 Created with the newly created event in the response body.
- On failure: Returns appropriate HTTP error payload.

Service Name: Get Admin Events Service

Description: Retrieves a list of all hackathon events that were created by a specific admin, this is done by using their specific UUID and matching it to the createdByUserId field.

Inputs:

- createdByUserId (UUID) - the unique identifier of the admin whose events are being retrieved.

Output:

- Returns a List<Event> array in the response body. Each event in the body is a valid event that satisfies the rules in the Create Event Service. If the admin has not yet created any events an empty array will be returned.

Usage / Interaction Rules:

- Client must send a GET request to the /api/admin/events.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 200 OK containing the JSON array of the matching event entities (or an empty array if no events could be found for an admin).
- On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.
- On failure: HTTP 400 if the id provided is malformed or not a valid UUID.

Service Name: Update Event Service

Description: Performs a complete overwrite of an existing hackathon event's configured data using the provided event identifier and request details.

Input:

- EventRequest object payload in the request body that contains:
    - Name (string) - The title of the event.
    - registrationKey (string) - The optional passkey for private events.
    - teamSizeLimit (short) - The maximum number of participants per team.
    - startDateTime (OffsetDateTime) - The starting time and date of the event.

- Duration (integer) - the total duration of an event in hours.
- Description (string) - Information about the specific hackathon event.
- Visibility (string) - The visibility of the hackathon event to the participants.
- Status (string) - The lifecycle of an event, it can be ACTIVE or INACTIVE as an example.

Output:

- Returns the updated and fully persistent Event entity object with the updated information matching that specified eventId.

Usage / Interaction Rules:

- Clients must send a PUT request to /api/admin/events/{id} where id is the UUID of the specific event that exists on the database.
- The event must already be present in the database.
- On success: Return HTTP 200 OK containing the updated Event entity representation inside the response body payload.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.
- On failure: HTTP 404 Not Found if the passed in eventId is not in the database or could not be found.

Service Name: Patch Event Status Service

Description: Performs a partial update on an existing hackathon event to modify its status, visibility or registrationKey if needed. The rest of the event stays unchanged.

Input:

- An EventRequest object payload in the request body that contains:
    - Visibility (string) - The new visibility state.
    - Status (string) - the new status state.
    - registrationKey (string) - the updated passkey to the event (this is optional and only required of the event visibility is private)

Output:

- Returns the modified and fully persistent Event entity object with the updated information matching that specified eventId.

Usage / Interaction Rules:

- Client must send a PATCH request to /api/admin/events/{id}/status where id represents the target events id.
- If the visibility is private then a registrationKey must be provided, otherwise it will automatically be set to null.
- On success: Return HTTP 200 OK containing the modified Event entity representation inside the response body payload.
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Service Name: Get Event Status Service

Description: Retrieves a lightweight summary containing only the current operational status and event visibility of a specific hackathon event.

Input:

- eventId (UUID) - the unique identifier of the specific event we are inspecting.

Output:

- Status (string) - the current status of the event we are inspecting.
- Visibility (String) - the current visibility state of the event we are inspecting.

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/events/{id}/status where id is the UUID of the specified event.
- The endpoint is restricted to authenticated users with the role admin (ROLE_ADMIN).
- On success: HTTP 200 OK containing the EventStatusResponse JSON payload
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Authorization

Service Name: Post Register User

Description: Registers a user by entering their credentials into the system. Assigns a Participant role by default.

Input:

- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Password (String) - Password of the user.

Output:

- Token (string) - JWT token.
- UserId (UUID) - IDof the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/register.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 409 Conflict if the user is already registered.

Service Name: Post Login User

Description: Login using saved email and password.

Input:

- Email (string) - Users email.
- Password (string) - Users password.

Output:

- Token (string) - JWT token.
- UserId (UUID) - IDof the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/login.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 404 Not Found if the user is not registered.

Scoring

Service Name: Score Submission Service

Description: Triggers scoring for a submission: runs the active solver against the submission's output file and persists score, status and logs. Safe to call again later (e.g. admin-triggered re-scoring after a solver hotfix) - it only overwrites this submission's own log, no other submission's log is touched.

Inputs**:**

- submissionId (long) - Identifier of the submission to score (in the URL path)

Outputs:

- submissionId (string) - The identifier of the submission being scored
- status (string) - The submission status ("QUEUED")
- recordId (string) - The queue record identifier for the scoring job

Usage / Interaction Rules:

- Clients must send a POST request to /api/scoring/submissions/{submissionId}/score
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 202 Accepted with submissionId, status, and recordId
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Team Submission History Service

Description: Full submission history for a team across all levels, most recent first. Used by the participant portal's "submission history" view.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)

Outputs:

- A list of SubmissionResponse objects, each containing:
    - submissionId (long) - Submission identifier
    - teamId (UUID) - Team identifier
    - levelId (short) - Level identifier
    - solverVersionId (long) - Solver version used
    - score (BigDecimal) - Score achieved

- status (string) - Submission status (QUEUED, PROCESSING, SCORED, FAILED)
- submittedAt (datetime) - Submission timestamp
- outputFileName (string) - Output filename
- sourceFileName (string) - Source archive filename
- scoringLog (ScoringLogResponse) - Scoring log (null in history view)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Team Level Submission History Service

Description: Submission history for a team, scoped to a single level, most recent first.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- levelId (short) - Identifier of the level (in the URL path)

Outputs:

- A list of SubmissionResponse objects (same structure as Get Team Submission History Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/levels/{levelId}/submissions
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Submission Detail Service

Description: Full feedback for a single submission, including score, status and its scoring log. Scoped to the owning team so a participant can't view another team's feedback by guessing IDs.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A SubmissionResponse object containing:
  - submissionId (long) - Submission identifier
  - teamId (UUID) - Team identifier
  - levelId (short) - Level identifier
  - solverVersionId (long) - Solver version used
  - score (BigDecimal) - Score achieved
  - status (string) - Submission status (QUEUED, PROCESSING, SCORED, FAILED)
  - submittedAt (datetime) - Submission timestamp
  - outputFileName (string) - Output filename
  - sourceFileName (string) - Source archive filename
  - scoringLog (ScoringLogResponse) - Scoring log with content

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the SubmissionResponse JSON payload
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Submission Log Service

Description: Just the scoring log for a single submission (score/status not included). Scoped to the owning team so a participant can't view another team's log by guessing IDs.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A ScoringLogResponse object containing:
    - submissionId (long) - Submission identifier
    - teamId (UUID) - Team identifier
    - eventId (UUID) - Event identifier
    - logPath (string) - Blob storage path of the log
    - scoredAt (datetime) - Scoring timestamp
    - logContent (string) - The content of the scoring log

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions/{submissionId}/log
- The request must include authentication (JWT)
- Admin or Participant role required

- On success, returns HTTP 200 OK with the ScoringLogResponse JSON payload
- On failure (e.g., log not found), returns HTTP 404 Not Found

Service Name: Admin Get Submission Detail Service

Description: Admin variant: full feedback for any submission regardless of team, for support/auditing.

Inputs:

- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A SubmissionResponse object (same structure as Get Submission Detail Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the SubmissionResponse JSON payload
- On failure, returns an appropriate HTTP error with a message

Service Name: Admin Get Submission Log Service

Description**:** Admin variant: just the scoring log for any submission regardless of team.

Inputs:

- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A ScoringLogResponse object (same structure as Get Submission Log Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/submissions/{submissionId}/log
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the ScoringLogResponse JSON payload
- On failure (e.g., log not found), returns HTTP 404 Not Found

Service Name: Admin Rescore Hackathon Service

Description: Re-enqueues every submission for all events under a hackathon (e.g. after a solver hotfix).

Inputs:

- hackathonId (UUID) - Identifier of the hackathon whose submissions should be rescored (in the URL path)

Outputs:

- hackathonId (string) - Identifier of the hackathon being rescored
- queuedCount (int) - Number of submissions that were enqueued
- status (string) - The operation status ("QUEUED")

Usage / Interaction Rules:

- Clients must send a POST request to /api/scoring/admin/hackathons/{hackathonId}/rescore
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 202 Accepted with hackathonId, queuedCount, and status
- On failure, returns an appropriate HTTP error with a message

Service Name: Admin Get Recent Submissions Service

Description: Retrieves recent submissions for admin monitoring.

Inputs:

- limit (int) - Maximum number of submissions to retrieve (in the URL path)

Outputs:

- A list of SubmissionResponse objects (same structure as Get Team Submission History Service, without scoring logs)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/recentsubmissions/{limit}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the JSON array of recent submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Level Leaderboard Service

Description: Retrieves the leaderboard for a specific level within an event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- levelId (short) - Identifier of the level (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects, each containing:
  - rank (int) - Ranking position
  - teamId (UUID) - Team identifier
  - teamName (string) - Name of the team
  - bestScore (BigDecimal) - Best score achieved
  - lastScoredAt (datetime) - Timestamp of the last scoring

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/levels/{levelId}/leaderboard
- The request must include authentication (JWT)

- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of leaderboard entries
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Event Leaderboard Service

Description: Retrieves the overall leaderboard for an event (all levels combined).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects (same structure as Get Level Leaderboard Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/leaderboard
- The request must include authentication (JWT)

- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of leaderboard entries
- On failure, returns an appropriate HTTP error with a message

Service Name: Event Leaderboard Update Stream Service

Description: Server-Sent Events stream for real-time leaderboard updates for an event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- Server-Sent Events (SSE) stream of leaderboard updates. Each event contains updated leaderboard data.

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/leaderboard/update
- The request must include authentication (JWT)
- Admin or Participant role required
- The endpoint produces MediaType.TEXT_EVENT_STREAM_VALUE
- On success, returns an SSE stream with leaderboard updates
- On failure, returns an appropriate HTTP error with a message

Leaderboard

Service Name: Get Level Leaderboard Service

Description:

- Retrieves the leaderboard for a specific level within an event. Ranks the teams according to their best scored submission for that level. The highest score will get the best rank (i.e. 1).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- levelId (Long) - Identifier of the level (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects, each containing:
    - Rank (int) - the team position on the leaderboard.
    - teamId (UUID) - identifier of the team.
    - teamName (string) - the name of the team.
    - bestScore (BigDecimal) - the best score achieved by the team this event
    - lastScoredAt (datetime) - Timestamp of when submission was scored, may be null.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/levels/{levelId}/leaderboard

- The request must include authentication (JWT)
- Admin or Participant role required
- Only submissions with status SCORED are included in the leaderboard calculation.
- If a team has not been scored yet, they automatically get a score of 0 on the leaderboard.
- Results are ordered by bestScore in descending order.
- On success, return HTTP 200 OK with a JSON array of leaderboard entries.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Leaderboard Service

Description:

- Retrieves the leaderboard for a specific event across all levels. Ranks the teams according to their best scored submission for that level. The highest score will get the best rank (i.e. 1).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- Rank (int) - the team position on the leaderboard.
- teamId (UUID) - identifier of the team.
- teamName (string) - the name of the team.
- bestScore (BigDecimal) - the best score achieved by the team this event
- lastScoredAt (datetime) - Timestamp of when submission was scored, may be null.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/leaderboard
- The request must include authentication (JWT)
- Admin or Participant role required

- Only submissions with status SCORED are included in the leaderboard calculation.
- The final best score is computed by summing the team's best score across all levels.
- If a team has not been scored yet, they automatically get a score of 0 on the leaderboard.
- Results are ordered by bestScore in descending order.
- On success, return HTTP 200 OK with a JSON array of leaderboard entries.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Level Leaderboard Service

Description:

- Provides SSE stream for real-time leaderboard update notifications. This allows the connected clients to be notified when the leaderboard might have changed after a submission has been graded.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- SSE event stream
- Each leaderboard-update event contains update metadata:
    - eventId (UUID) - identifier of the event whose leaderboard has been updated.
    - levelId (Long) - identifier of the level affected by the scored submission.
    - teamId (UUID) - Identifier of the team whose submission was scored.
    - submissionId (Long) - Identifier of the scored submission.
    - OccurredAt (datetime) - Timestamp of when the update event was issued.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/leaderboard/update
- The request must include authentication (JWT)

- Admin or Participant role required
- The endpoint produces: MediaType.TEXT_EVENT_STREAM_VALUE
- The connection remains open as long as the client is subscribed to leaderboard updates
- When a submission is successfully scored, the backend sends a leaderboard-update event to connect clients for the relevant event
- SSE acts as a notification that the leaderboard has changed
- After receiving the event, the client should refresh the leaderboard by calling: GET /api/scoring/events/{eventId}/leaderboard
- On Success, returns SSE stream with leaderboard update notifications
- On Failure, returns an appropriate HTTP error with a message

Service Name: Create Hackathon Service

Description:

- Creates a new hackathon that participants can register for and administrators can manage.

Inputs:

- HackathonRequest - JSON request body containing the hackathon details.

Outputs:

- Hackathon - The newly created hackathon object.

Usage / Interaction Rules:

- Client must send a POST request to:
  /api/hackathon
- The request must include authentication (JWT).
- Admin role required.
- Request body must contain valid hackathon information.
- On success, returns HTTP 200 OK with the created hackathon.
- On failure, returns an appropriate HTTP error message.

Service Name: Get All Hackathons Service

Description:

- Retrieves all hackathons available in the system.

Inputs:

- None

Outputs:

- Hackathon[] - List of hackathons.

Usage / Interaction Rules:

- Client must send a GET request to:
  /api/hackathon
- Authentication is not required.
- Returns HTTP 200 OK with a JSON array of hackathons.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Service

Description:

- Retrieves a specific hackathon by its identifier.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (URL path).

Outputs:

- Hackathon - Requested hackathon.

Usage / Interaction Rules:

- Client must send a GET request to:
  /api/hackathon/{hackathonId}
- Authentication is not required.
- Returns HTTP 200 OK with the hackathon.
- Returns an appropriate HTTP error if the hackathon cannot be found.

Service Name: Update Hackathon Service

Description:

- Update Hackathon Service

Inputs:

- hackathonId (UUID) - Identifier of the hackathon.
- HackathonRequest- Updated hackathon information.

Outputs:

- Hackathon- Updated hackathon.

Usage / Interaction Rules:

- Client must send a PUT request to:
  /api/hackathon/{hackathonId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the updated hackathon.
- On failure, returns an appropriate HTTP error message.

Service Name: Delete Hackathon

Description:

- Deletes an existing hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- None

Usage / Interaction Rules:

- Client must send a DELETE request to: /api/hackathon/{hackathonId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 204 No Content on success.
- On failure, returns an appropriate HTTP error message.

Service Name: Create Event

Description:

- Creates a new event for a specified hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.
- EventRequest- Event details.

Outputs:

- Event – Newly created event.

Usage / Interaction Rules:

- Client must send a POST request to: /api/hackathon/{hackathonId}/events
- The request must include authentication (JWT).
- Admin role required.
- The supplied hackathon ID is associated with the event before creation.
- Returns HTTP 200 OK with the created event.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Events Service

Description:

- Retrieves all events belonging to a specific hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- Event[]- List of events.

Usage / Interaction Rules:

- Client must send a GET request to: /api/hackathon/{hackathonId}/events
- Authentication is not required.
- Returns HTTP 200 OK with a JSON array of events.
- On failure, returns an appropriate HTTP error message.

Service Name: Create Level

Description:

- Creates a new competition level for a hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.
- LevelRequest- Level details.

Outputs:

- Level- Newly created level.

Usage / Interaction Rules:

- Client must send a POST request to: /api/hackathons/{hackathonId}/levels
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the created level.
- On failure, returns an appropriate HTTP error message.

Service Name: Update Level

Description:

- Updates an existing competition level.

Inputs:

- levelId (short)- Identifier of the level.
- LevelRequest- Updated level details.

Outputs:

- Level- Updated level.

Usage / Interaction Rules:

- Client must send a PUT request to: /api/levels/{levelId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the updated level.
- On failure, returns an appropriate HTTP error message.

Service Name: Delete Level

Description:

- Deletes a competition level.

Inputs:

- levelId (short)- Identifier of the level.

Outputs:

- None

Usage / Interaction Rules:

- Client must send a DELETE request to: /api/levels/{levelId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 204 No Content on success.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Levels

Description:

- Retrieves all competition levels associated with a hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- Level[]- List of competition levels.

Usage / Interaction Rules:

- Client must send a GET request to: /api/hackathons/{hackathonId}/levels
- Authentication is not required.
- Returns HTTP 200 OK with a JSON array of levels.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Level

Description:

- Retrieves the details of a specific competition level.

Inputs:

- levelId (short)- Identifier of the level.

Outputs:

- Level- Requested competition level.

Usage / Interaction Rules:

- Client must send a GET request to:
  /api/levels/{levelId}
- Authentication is not required.
- Returns HTTP 200 OK with the requested level.
- Returns an appropriate HTTP error message if the level cannot be found.